Entrega 3: Desarrollo, Entrenamiento y Experimentación del Modelo

Fundamentos de Inteligencia Artificial, Proyecto Final

Fecha de entrega: 11 de noviembre de 2026 (fin del Sprint 3)
Nombre del proyecto: ClimateAI
Equipo / número de grupo: Equipo ClimateAI, grupo LITIID71
Repositorio: https://github.com/HectorPartido/ClimateAI
Integrantes y roles:

1. Hector Partido Flores: Coordinación (Project Manager); PR 0 y PR 5
2. Anthony Oziel Ramos López: Líder de Datos y Preprocesamiento; PR 1
3. Hector Martínez Sánchez: Líder de Modelado (grupos de países); PR 2
4. Ismael Alexander Hidalgo Aguilando: Líder de Modelado (lógica difusa); PR 3
5. Gildardo Emmanuel González García: Líder de Pipeline e Integración; PR 4

---

1. Selección y Justificación Técnica de los Modelos

ClimateAI asigna a cada país una prioridad climática de **adaptación** y otra de **mitigación** (0 a 100, con los niveles Baja, Media, Alta y Crítica) a partir de 6 indicadores calculados con Berkeley Earth y Our World in Data, con corte en 2012 (I1 tendencia de calentamiento, I2 anomalía, I3 variabilidad, I4 emisiones acumuladas, I5 participación mundial e I6 crecimiento de emisiones). El motor tiene dos etapas, y en cada una se compararon varios modelos sobre los mismos 41 países de prueba de la Entrega 2.

A. Definición de la Estrategia de Modelado

☒ **Estrategia comparativa (adoptada):** en la etapa de grupos se comparan K-means sin PCA, PCA + K-means y PCA + mezcla gaussiana; en la etapa de prioridad se comparan una regla de cuartiles y dos versiones del sistema difuso Mamdani.

☐ **Estrategia baseline vs. modelo optimizado.** No se eligió como estrategia principal, aunque dentro del sistema difuso la versión 2 de las reglas es una optimización de la versión 1.

B. Justificación de la Elección

Modelo / Algoritmo	Paradigma / Familia	Justificación técnica (¿por qué resuelve el problema?)
Modelo 1, etapa 1 (candidato): PCA de 2 componentes + K-means, k = 4 (Exp 2)	No supervisado: reducción de dimensionalidad y agrupamiento	No existe una etiqueta de "perfil climático" por país, así que se agrupa sin supervisión. Con los 6 indicadores en z-score la silueta en Train queda en 0.318, menor que la meta de 0.35; el PCA conserva el 76.5 % de la varianza y elimina la redundancia (I1 con I2, I4 con I5). k = 4 se fijó desde la Entrega 1 para separar los 4 perfiles de calentamiento y emisiones.

Modelo / Algoritmo	Paradigma / Familia	Justificación técnica (¿por qué resuelve el problema?)
Modelo 1, etapa 2 (candidato): Lógica difusa Mamdani, reglas versión 2 (Exp 6)	Sistema experto basado en reglas difusas (mínimo, máximo y centroide)	Tampoco hay una "prioridad correcta" registrada para entrenar un modelo supervisado. La lógica difusa da transiciones suaves entre niveles y reglas legibles: `predecir()` devuelve qué reglas se activaron y con qué fuerza, lo que permite explicar cada resultado con al menos 2 indicadores (CA6).
Modelo 2, etapa 1 (baseline y comparativo): K-means con los 6 indicadores (Exp 1) y PCA + mezcla gaussiana (Exp 3)	No supervisado: agrupamiento por distancia y por probabilidad	El Exp 1 muestra si el PCA realmente aporta. La mezcla gaussiana permite grupos de forma elíptica y tamaños distintos, para comprobar si una forma más flexible separa mejor que K-means.
Modelo 2, etapa 2 (baseline y comparativo): Regla de cuartiles de un indicador (Exp 4) y Mamdani versión 1 (Exp 5)	Regla estadística simple y sistema difuso	La regla de cuartiles (adaptación con I1, mitigación con I5) es la línea base de la Entrega 2: sirve para saber si las 12 a 15 reglas difusas aportan algo frente a un solo indicador. La versión 1 de las reglas es el diseño original de 15 reglas por sistema.

Por qué no un modelo supervisado (Random Forest, SVM): necesitaría una etiqueta por país; crearla a mano para 201 países sería circular, porque el modelo aprendería el criterio de quien etiquetó. Por eso las etiquetas del experto se usan solo para **validar**, nunca para entrenar.

---

2. Pipeline de Datos y Estructura en el Repositorio

Todas las rutas salen de `src/config.py` y se calculan desde la raíz del repositorio, así que el proyecto corre igual en cualquier computadora después de clonarlo. Se reprodujo desde cero en las computadoras de los integrantes y en un entorno limpio de Linux.

A. Ubicación de Archivos en el Repositorio

- **Script de entrenamiento:** `src/models/train.py`, que ejecuta `src/models/clustering.py` (Exp 1 a 3) y `src/models/fuzzy.py` (Exp 4 a 6).
- **Script del pipeline de preprocesamiento:** `src/data/preprocess.py` (limpieza, emparejamiento de nombres de país, indicadores I1 a I6 y partición 160/41 con `random_state = 42`).
- **Modelos serializados:** `models/kmeans_k4.joblib` (candidato), `models/gmm_k4.joblib` (comparativo), `models/difuso.json` (percentiles de Train y reglas de los 2 sistemas) y `models/grupos.json` (nombre y centroide de cada grupo).
- **Transformadores guardados:** `models/scaler.joblib` (StandardScaler ajustado solo con Train) y `models/pca.joblib`.
- **Inferencia y pruebas:** `src/models/predict.py` y `tests/test_predict.py` (22 pruebas).
- **Validación con el experto:** `src/data/hoja_experto.py`, `data/expert/respuestas_docente.xlsx` y `data/expert/niveles_experto.csv`.
- **Resultados:** `reports/experimentos.csv` (matriz), `reports/resumen.json`, `reports/niveles_pais.csv`, `reports/figuras/` y `logs/entrenamiento.log`.
- **Archivo de dependencias:** `requirements.txt` (numpy, pandas, scikit-learn, scikit-fuzzy, joblib, matplotlib, openpyxl y pytest).

B. Comando de Reproducibilidad en una Sola Línea

Desde los 3 CSV originales, el comando genera los indicadores, la partición, los 6 experimentos, los modelos, las figuras, la matriz de experimentación y el registro en unos 15 a 60 segundos según la computadora:

```
Shell
python src/models/train.py --raw data/raw --output models/
```

Pasos completos para clonar y reproducir:

```
Shell
git clone https://github.com/HectorPartido/ClimateAI.git
cd ClimateAI
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python src/models/train.py --raw data/raw --output models/
python src/models/predict.py       # demo de predecir() con México
python -m pytest -q                # 22 pruebas
```

---

3. Matriz de Experimentación y Ajuste de Hiperparámetros

Todos los experimentos usan la partición de la Entrega 2 (`train_test_split`, 20 % de prueba, `random_state = 42`): 160 países en Train y 41 en Test. En la etapa 1 la métrica principal es la silueta en Test (meta 0.30; en Train la meta es 0.35). En la etapa 2 es la concordancia de nivel con el experto en 20 países de Test (meta de 16 de 20 por sistema, OE3 y CA5). Los tiempos son de una ejecución y cambian según la computadora.

Variante	Modelo	Hiperparámetros clave	Métrica principal (Test)	Métrica secundaria (Test)	Tiempo entren. / inferencia	Diagnóstico
Exp 1: Baseline	K-means sobre los 6 indicadores en z-score	k = 4, n_init = 10, distancia euclidiana	Silueta: 0.368	Davies-Bouldin: 0.941	0.066 s / 0.34 ms	Subajuste: 0.318 en Train, menor que la meta de 0.35
Exp 2: Candidato	PCA + K-means	PCA de 2 componentes (76.5 % de varianza), k = 4, n_init = 10	Silueta: 0.382	Davies-Bouldin: 0.814	0.009 s / 0.42 ms	Óptimo: 0.407 en Train y 0.382 en Test; ARI de estabilidad 0.822
Exp 3: Alternativo	PCA + mezcla gaussiana (GMM)	4 componentes, covarianza completa, n_init = 10	Silueta: 0.328	Davies-Bouldin: 1.645	0.059 s / 0.24 ms	Subajuste: 0.329 en Train
Exp 4: Baseline	Regla de cuartiles de un indicador	Adaptación con I1, mitigación con I5; cortes P25, P50 y P75 de Train	Concordancia con experto: adaptación 9/20 (45 %), mitigación 4/20 (20 %)	F1 macro: 0.38 y 0.21	< 0.01 s / 0.03 ms por país	Línea base: no alcanza la meta; explica con un solo indicador

Variante	Modelo	Hiperparámetros clave	Métrica principal (Test)	Métrica secundaria (Test)	Tiempo entren. / inferencia	Diagnóstico
Exp 5: Alternativo	Mamdani, reglas versión 1	15 reglas por sistema (malla 3 x 3 + 6 ajustes); P20, P50 y P80 de Train; mínimo, máximo y centroide	12/20 (60 %) y 4/20 (20 %)	F1 macro: 0.57 y 0.23	< 0.01 s / 0.96 ms por país	Descartado: más riesgo puede bajar la prioridad (13 cambios de nivel)
Exp 6: Candidato	Mamdani, reglas versión 2	12 reglas por sistema (malla 3 x 2 con "no alto" + 6 ajustes); mismas funciones y operadores	13/20 (65 %) y 4/20 (20 %)	F1 macro: 0.62 y 0.23	< 0.01 s / 0.80 ms por país	Subajuste en mitigación (sesgo de un nivel); adaptación coherente y a un nivel o menos en 20/20

Líneas base de la etapa 1 (silueta en Train): partición aleatoria en 4 grupos −0.061 y 4 grupos por cuartiles de I4 0.129. Los 4 grupos del Exp 2 son: 1 Grandes emisores con calentamiento bajo (37 países), 2 Bajo impacto con calentamiento leve (84), 3 Emisores históricos con calentamiento alto (56) y 4 Bajo impacto con calentamiento alto (24).

Validación con el experto (20 países de Test)

El docente asignó los niveles a ciegas: vio los indicadores y su lugar frente a los 201 países, pero no las reglas ni los resultados del modelo. Además de la concordancia exacta se midió cuántos países quedan a un nivel o menos, la kappa ponderada cuadrática (acuerdo para niveles ordenados) y el sesgo (nivel del modelo menos nivel del experto, en promedio).

Experimento	Adaptación exacta	A ±1 nivel	Kappa	Sesgo	Mitigación exacta	A ±1 nivel	Kappa / sesgo
Exp 4: cuartiles	9/20 (45 %)	19/20	0.75	+0.40	4/20 (20 %)	13/20	0.41 / +1.15
Exp 5: Mamdani v1	12/20 (60 %)	20/20	0.81	+0.20	4/20 (20 %)	16/20	0.50 / +1.00
Exp 6: Mamdani v2	13/20 (65 %)	20/20	0.83	+0.25	4/20 (20 %)	16/20	0.50 / +1.00

Análisis Crítico de los Resultados

1. ¿Qué modelo resultó ganador y por qué?

Etapa 1: PCA + K-means (Exp 2). Es el único que supera la meta de silueta en Train (0.407) y la mantiene en Test (0.382), con el menor índice de Davies-Bouldin (0.814). Con los 6 indicadores (Exp 1) las distancias se diluyen: I1 e I2 miden casi lo mismo, igual que I4 e I5, y esas parejas pesan doble, mientras I6 aporta poco (es el indicador con menor carga en los dos componentes). El PCA resume todo en dos ejes con sentido: PC1 (52.3 %) mide la magnitud y PC2 (24.2 %) separa responsabilidad (emisiones) de exposición (calentamiento). La mezcla gaussiana (Exp 3) maximiza la verosimilitud, no la separación: forma componentes alargados y traslapados, por eso su Davies-Bouldin es el doble (1.645).

Etapa 2: Mamdani versión 2 (Exp 6). En adaptación es el mejor: 13 de 20 exactos contra 9 de la regla de cuartiles, los 20 a un nivel o menos y kappa de 0.83. Además es coherente (más riesgo nunca baja de nivel) y explica cada resultado con 2 indicadores en los 82 casos de

prueba, mientras la línea base usa uno solo. La versión 1 (Exp 5) se descartó por un problema matemático de la agregación por máximo: dos celdas vecinas de la malla con el mismo nivel pesan solo 0.5 a medio camino, el centroide se va hacia las reglas de ajuste y el puntaje podía bajar al subir un indicador de riesgo (93 retrocesos y 13 cambios de nivel en 10,086 pasos). La versión 2 fusiona esas celdas con el término "no alto" (1 − alto, que equivale a bajo + medio) y corrige 2 reglas que contradecían a la malla: quedan 9 retrocesos de 1.1 puntos como máximo y ninguno cambia de nivel.

Mitigación no alcanza la meta en ningún experimento (4 de 20). El error es sistemático: el modelo queda un nivel arriba del experto en 16 países y nunca abajo. La causa está en la escala, no en las reglas: las funciones de pertenencia de I4 e I5 usan percentiles por **número de países**, y como las emisiones están muy concentradas (la mitad de los países de Train aporta menos de 0.021 % de las emisiones mundiales), un país con 0.04 % ya cae en "medio". El experto juzga por el **peso real en el mundo**: debajo de 0.1 % lo considera marginal aunque sus emisiones crezcan mucho. Por ejemplo:

País	Parte de las emisiones mundiales	Crecimiento	Modelo (Exp 6)	Experto
Guatemala	0.036 %	+39 %	Alta	Baja
Myanmar	0.038 %	+53 %	Alta	Baja
Cuba	0.088 %	+17 %	Alta	Baja
Nigeria	0.312 %	+14 %	Crítica	Media
Taiwán	0.850 %	+36 %	Crítica	Alta
Arabia Saudita	1.500 %	+69 %	Crítica	Crítica
India	4.633 %	+68 %	Crítica	Crítica

En adaptación los 7 errores son de un solo nivel; el caso más claro es Bélgica (Alta en el modelo y Media para el experto), donde el experto pesa más la capacidad económica para adaptarse.

2. Detección y mitigación del sobreajuste

- **Etapa 1, Train contra Test:** Exp 2 pasa de 0.407 a 0.382 de silueta (caída de 0.025, menor que el límite de 0.10 que definimos como sobreajuste). El Exp 1 incluso mejora en Test (0.318 a 0.368) y el Exp 3 se mantiene (0.329 a 0.328). La estabilidad (ARI entre el modelo de Train y uno ajustado con todos los datos) es de 0.822 para el candidato.
- **Regularización en la etapa 1:** la reducción con PCA a 2 componentes funciona como regularización, porque descarta la varianza de ruido; el escalador y el PCA se ajustan solo con Train y `n_init = 10` evita depender de una sola inicialización.
- **Etapa 2:** el sistema difuso no aprende pesos; con Train solo se calculan los percentiles P20, P50 y P80. El riesgo de sobreajuste está en ajustar las reglas mirando a los 20 países del experto: **no se hizo**, porque son el conjunto de prueba y la concordancia dejaría de ser una medida honesta.
- **Simplificación como regularización:** pasar de 15 a 12 reglas por sistema, fusionando celdas con el mismo nivel, hizo el modelo más simple y coherente sin perder concordancia (12 a 13 de 20 en adaptación).
- **Sugerencia del docente:** con análisis estadístico se puede etiquetar a la mitad de los países (unos 100), como haría un experto. Con eso se calibraría la escala de mitigación

con una mitad y se validaría con la otra y con los 20 del docente, sin tocar el conjunto de prueba. Es el plan para la Entrega 4 (sección 6).

---

4. Evidencias de Ejecución Funcional

A. Registro de Consola del Entrenamiento

Salida de `python src/models/train.py --raw data/raw --output models/` y `python -m pytest -q` en la computadora del Project Manager (también queda en `logs/entrenamiento.log`):

```
Consola
[INFO] ClimateAI, Entrega 3: entrenamiento y experimentación (random_state = 42)
[INFO] Versiones: python 3.14.0, numpy 2.5.4, pandas 3.1.1, scikit-learn 1.9.1, scikit-fuzzy 0.5.0,
matplotlib 3.11.2
[INFO] Cargando datasets desde data/raw...
[INFO] Temperatura por país: 577,462 filas leídas, 197,599 descartadas (sin dato o incertidumbre > 1 °C)
[INFO] Emisiones de CO2: 29,384 filas leídas
[INFO] Emparejamiento: 203 entidades de Berkeley Earth con serie en Our World in Data
[INFO] Indicadores I1 a I6 para 201 países | Train: 160 | Test: 41 (random_state=42)
[SUCCESS] Indicadores y partición guardados en data/processed
[INFO] Entrenando agrupamiento (Exp 1 a 3)...
[INFO] Exp 1 (modelo base): K-means k=4 sobre los 6 indicadores en z-score...
[INFO] Exp 2 (candidato): z-score, PCA de 2 componentes y K-means k=4...
[INFO] Exp 3 (alternativo): mezcla gaussiana de 4 componentes sobre el mismo PCA...
[INFO] Exp 1: Baseline | Silueta Train: 0.318 | Silueta Test: 0.368 | Davies-Bouldin Test: 0.941 |
Subajuste: los grupos no quedan bien separados (silueta en Train < 0.35)
[INFO] Exp 2: Candidato | Silueta Train: 0.407 | Silueta Test: 0.382 | Davies-Bouldin Test: 0.814 |
Óptimo: cumple la meta y la separación se mantiene en Test
[INFO] Exp 3: Alternativo | Silueta Train: 0.329 | Silueta Test: 0.328 | Davies-Bouldin Test: 1.645 |
Subajuste: los grupos no quedan bien separados (silueta en Train < 0.35)
[INFO] Líneas base (silueta en Train): partición aleatoria -0.061, cuartiles de I4 0.129
[INFO] Grupo 1, Grandes emisores con calentamiento bajo: 37 países (9 en Test)
[INFO] Grupo 2, Bajo impacto con calentamiento leve: 84 países (15 en Test)
[INFO] Grupo 3, Emisores históricos con calentamiento alto: 56 países (14 en Test)
[INFO] Grupo 4, Bajo impacto con calentamiento alto: 24 países (3 en Test)
[SUCCESS] Modelos guardados en models: scaler.joblib, pca.joblib, kmeans_k4.joblib, gmm_k4.joblib y
grupos.json
[INFO] Ajustando los sistemas difusos con los percentiles de Train y evaluando en Test (Exp 4 a 6)...
[INFO] Exp 4: Baseline | Concordancia con experto: Ada. 9/20 (45%), Mit. 4/20 (20%) | F1 macro: Ada.
0.38, Mit. 0.21 | Línea base: no alcanza la meta del 80 %
[INFO] Exp 5: Alternativo | Concordancia con experto: Ada. 12/20 (60%), Mit. 4/20 (20%) | F1 macro: Ada.
0.57, Mit. 0.23 | Subajuste: las reglas no capturan el criterio del experto (menos de 80 %); además, más
riesgo puede bajar la prioridad
[INFO] Exp 6: Candidato | Concordancia con experto: Ada. 13/20 (65%), Mit. 4/20 (20%) | F1 macro: Ada.
0.62, Mit. 0.23 | Subajuste: las reglas no capturan el criterio del experto (menos de 80 %)
[INFO] Exp 5 adaptacion: cobertura 1000 combinaciones (fuerza dominante mínima 0.54, 0 no concluyentes)
| coherencia 65 retrocesos en 5043 pasos (9 bajan de nivel) | acuerdo con Exp 4 73 %
[INFO] Exp 5 mitigacion: cobertura 1000 combinaciones (fuerza dominante mínima 0.54, 0 no concluyentes)
| coherencia 28 retrocesos en 5043 pasos (4 bajan de nivel) | acuerdo con Exp 4 83 %
[INFO] Exp 6 adaptacion: cobertura 1000 combinaciones (fuerza dominante mínima 0.54, 0 no concluyentes)
| coherencia 0 retrocesos en 5043 pasos (0 bajan de nivel) | acuerdo con Exp 4 68 %
[INFO] Exp 6 mitigacion: cobertura 1000 combinaciones (fuerza dominante mínima 0.54, 0 no concluyentes)
| coherencia 9 retrocesos en 5043 pasos (0 bajan de nivel) | acuerdo con Exp 4 83 %
[SUCCESS] Sistemas difusos guardados en models/difuso.json
[SUCCESS] Matriz de experimentación guardada en reports/experimentos.csv
[INFO] Probando predecir() con los 201 países y los 2 sistemas...
[INFO] predecir(): 402 llamadas, 0 no concluyentes, tiempo promedio 0.93 ms (máximo 1.13 ms; meta <=
1000 ms)
```

```
[SUCCESS] Niveles de los 201 países guardados en reports/niveles_pais.csv
[SUCCESS] Entrenamiento terminado en 14.4 s. Figuras en reports/figuras y registro en
logs/entrenamiento.log
...................... [100%]
22 passed in 1.58s
```

B. Matriz de Confusión y Curva de Convergencia

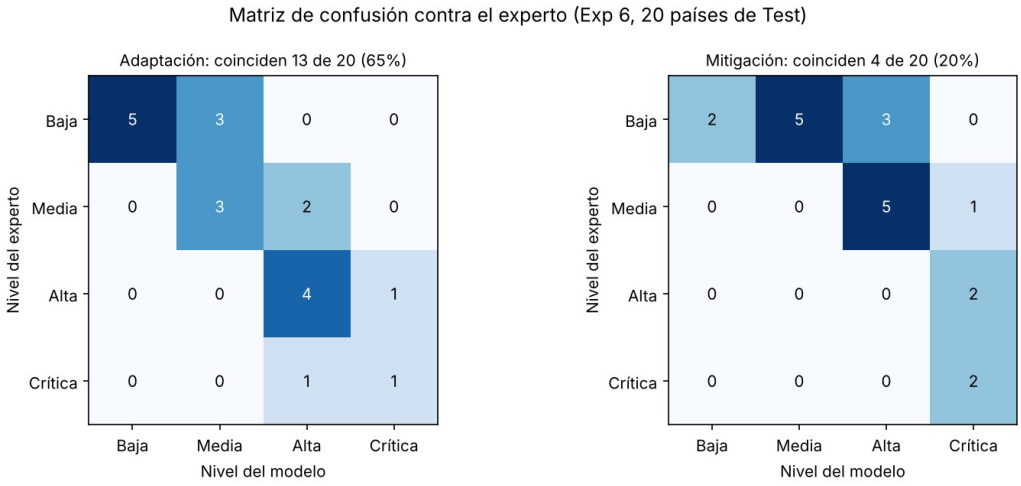


Figura 1. Matriz de confusión del modelo candidato (Exp 6) contra el experto en 20 países de Test. En mitigación todos los errores quedan arriba de la diagonal.

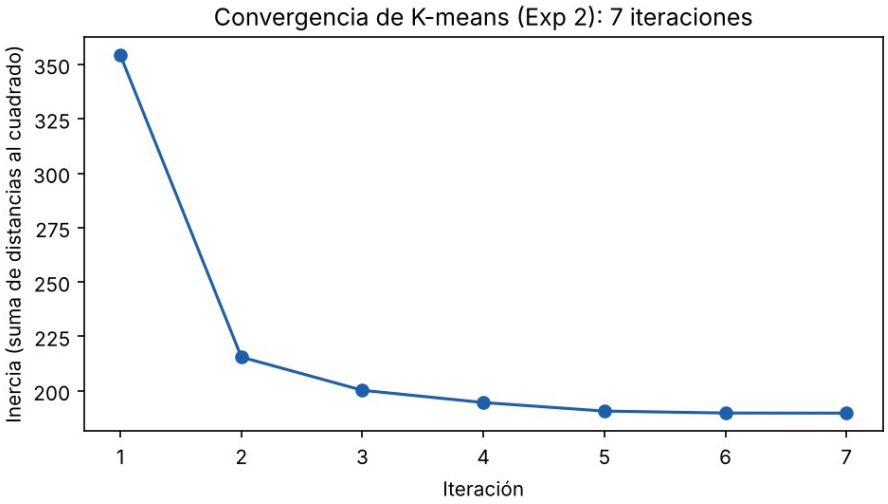


Figura 2. Curva de convergencia de K-means (inercia por iteración) del modelo candidato.

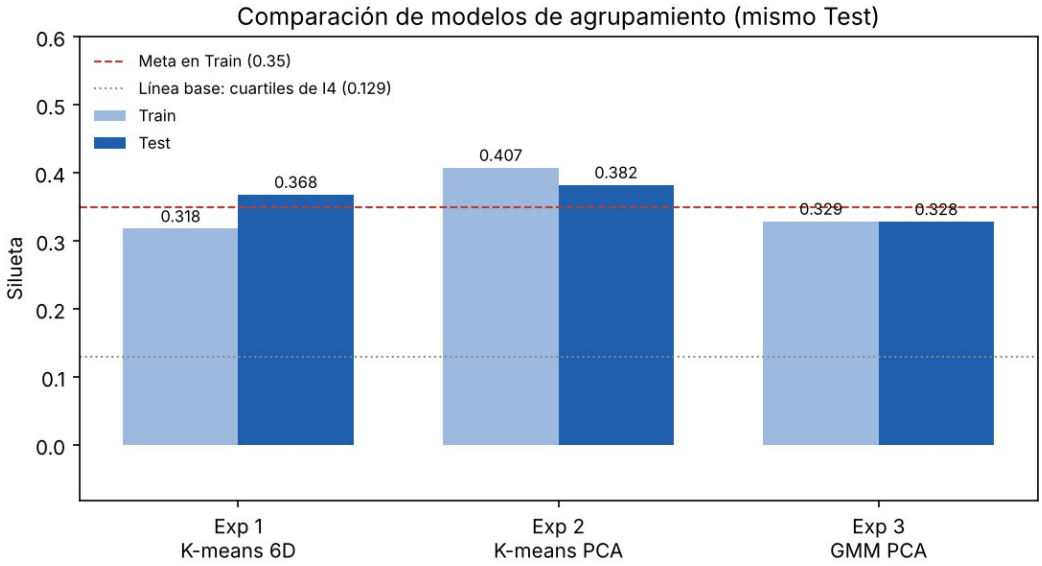


Figura 3. Silueta en Train y Test de los Exp 1 a 3, con la meta y la línea base aleatoria.

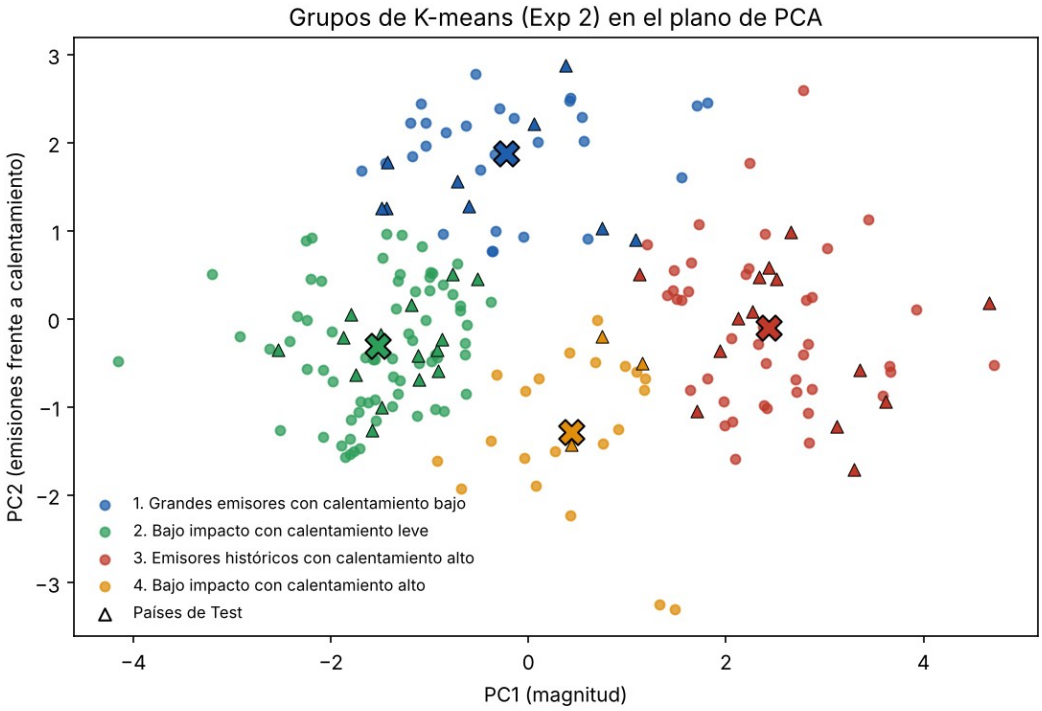


Figura 4. Los 4 grupos en el plano de los 2 componentes principales (círculos: Train; triángulos: Test; X: centroides).

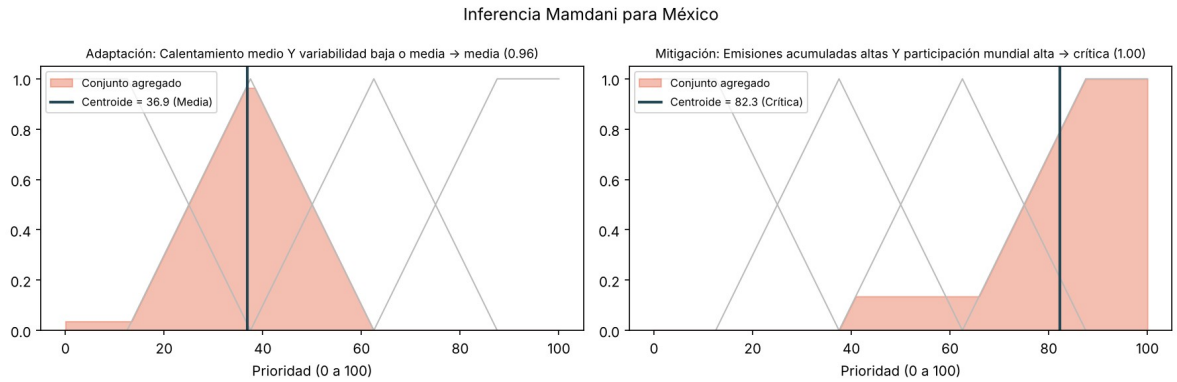


Figura 5. Inferencia Mamdani para México: conjunto agregado y centroide en adaptación y mitigación.

---

5. Función de Inferencia Desacoplada (predecir)

Vive en `src/models/predict.py` (la Entrega 2 la ubicaba en `src/modelo.py`; se movió a la estructura que pide esta plantilla). Carga los modelos una sola vez, valida la entrada, aplica el mismo preprocesamiento del entrenamiento y devuelve las 4 llaves del contrato de la Entrega 2 más las llaves extra que usará el tablero. Las 22 pruebas de `tests/test_predict.py` revisan el contrato, los errores de entrada, el tiempo, la coherencia y que los 201 países reciban nivel y explicación.

A. Código Fuente de la Función de Inferencia

Python (fragmento de src/models/predict.py)

```
def validar(datos_entrada) -> None:
    if not isinstance(datos_entrada, dict):
        raise ValueError("La entrada debe ser un diccionario con I1 a I6 y sistema")
    for llave, (minimo, maximo) in RANGOS.items():
        if llave not in datos_entrada:
            raise ValueError(f"Falta la llave {llave}")
        valor = datos_entrada[llave]
        if isinstance(valor, bool) or not isinstance(valor, Real) or not np.isfinite(valor):
            raise ValueError(f"{llave} debe ser un número (se recibió {valor!r})")
        if not minimo <= valor <= maximo:
            raise ValueError(f"{llave} = {valor} fuera de rango [{minimo}, {maximo}]")
    if datos_entrada.get("sistema") not in SISTEMAS:
        raise ValueError("sistema debe ser 'adaptacion' o 'mitigacion'")


def predecir(datos_entrada: dict) -> dict:
    """Prioridad climática (0 a 100) con lógica difusa Mamdani y grupo con PCA + K-means."""
    inicio = time.perf_counter()
    validar(datos_entrada)
    a = _artefactos()      # carga los modelos una sola vez
    valores = {c: float(datos_entrada[c]) for c in RANGOS}
    sistema = datos_entrada["sistema"]

    # grupo: mismo preprocesamiento que en el entrenamiento (log10, z-score de Train y PCA)
    etiqueta = int(a["kmeans"].predict(
        a["pca"].transform(a["scaler"].transform(preparar_matriz(valores))))[0])
    gid = a["grupos"]["etiqueta_a_grupo"][str(etiqueta)]
    grupo = {"id": int(gid), "nombre": a["grupos"]["grupos"][str(gid)]["nombre"]}

    # prioridad: sistema difuso del tipo pedido
    r = inferir(valores, sistema, a["difuso"])
```

```
    texto = explicar(valores, sistema, r, a["difuso"])
    estado = "no_concluyente" if r["no_concluyente"] else "ok"

    return {
        "prediccion": r["puntaje"],
        "clase_etiqueta": r["nivel"],
        "confianza_o_score": r["fuerza_maxima"],
        "tiempo_ms": round((time.perf_counter() - inicio) * 1000, 2),
        "grupo_kmeans": grupo,
        "explicacion": texto,
        "reglas_activadas": [{"regla": x["regla"], "fuerza": x["fuerza"]}
                             for x in r["reglas_activadas"]],
        "conjunto_agregado": r["conjunto_agregado"],
        "estado": estado,
    }
```

B. Prueba Unitaria de Entrada y Salida (Demo de Inferencia)

Llamada real con los valores de México (corte 2012), con `python src/models/predict.py`. El conjunto agregado tiene 1,001 puntos; aquí se recorta.

- **Ejemplo de entrada:**

```JSON
{
  "pais": "Mexico",
  "I1": 0.216,
  "I2": 0.551,
  "I3": 0.233,
  "I4": 16.428,
  "I5": 1.483,
  "I6": 25.9,
  "sistema": "mitigacion"
}
```

- **Salida real retornada por el modelo:**

```JSON
{
  "prediccion": 82.3,
  "clase_etiqueta": "Crítica",
  "confianza_o_score": 1.0,
  "tiempo_ms": 3.25,
  "grupo_kmeans": {"id": 1, "nombre": "Grandes emisores con calentamiento bajo"},
  "explicacion": "Su prioridad de mitigación es crítica (82 de 100) porque ...",
  "reglas_activadas": [
    {"regla": "Emisiones acumuladas altas Y participación mundial alta -> crítica",
     "fuerza": 1.0},
    {"regla": "Emisiones acumuladas altas Y crecimiento positivo -> crítica", "fuerza": 0.647},
    {"regla": "Participación mundial alta Y crecimiento positivo -> crítica", "fuerza": 0.647},
    {"regla": "Participación mundial alta Y crecimiento estable -> alta", "fuerza": 0.137}
  ],
  "conjunto_agregado": {"x": [0.0, 0.1, 0.2, "...", 100.0],
                        "mu": [0.0, 0.0, 0.0, "...", 1.0],
                        "centroide": 82.3},
  "estado": "ok"
}
```

La explicación completa que devuelve es: "Su prioridad de mitigación es crítica (82 de 100) porque ha emitido 16.4 Gt de CO_2 en su historia (más que el 93 % de los países) y aporta el 1.48 % de las emisiones mundiales (más que el 95 % de los países)."

Con `"sistema": "adaptacion"` la misma entrada da 37.0 (Media), con la regla "Calentamiento medio Y variabilidad baja o media -> media" (fuerza 0.967). En las 402 llamadas de prueba (201 países por 2 sistemas) no hubo resultados no concluyentes y el tiempo promedio fue de alrededor de 1 a 5 ms, muy por debajo de la meta de 1,000 ms. Si falta una llave, un valor no es número o está fuera de rango, `predecir()` lanza `ValueError`.

---

6. Reporte de Estado del Proyecto (Sprint Review para el Project Manager)

1. **Porcentaje global de avance del motor de IA: 85 %.** Están terminados el preprocesamiento, los 6 experimentos, los modelos serializados, `predecir()` con sus pruebas y la validación con el experto. Falta recalibrar la mitigación para cumplir la meta de concordancia (OE3).
2. **Semáforo del Sprint: con retraso moderado.** El motor está completo y reproducible, pero la métrica principal no alcanza la meta: adaptación 13 de 20 (cerca) y mitigación 4 de 20 (lejos).
3. **Bloqueos técnicos actuales:**
   - La escala de mitigación usa percentiles por número de países y sobrestima a los países que casi no emiten (sesgo de un nivel en 16 de 20).
   - Solo hay 20 países con nivel de experto: cada país vale 5 puntos de concordancia y no alcanzan para calibrar sin tocar el conjunto de prueba.
   - El pipeline y la función de inferencia no tienen bloqueos.
4. **Plan de acción para la Entrega 4:**
   - Conectar `predecir()` al tablero en Streamlit (Panorama, Grupos de países y Simulador de prioridad) y generar las gráficas comprometidas en la Entrega 2, con las llaves extra `reglas_activadas` y `conjunto_agregado`.
   - Recalibrar la mitigación con el peso real en las emisiones mundiales (por ejemplo, "alto" para los países que juntos suman el 80 % de las emisiones) y hacer que el crecimiento solo cuente cuando el país ya aporta una parte relevante.
   - Seguir la sugerencia del docente: etiquetar con análisis estadístico a la mitad de los países (unos 100), calibrar con una mitad, validar con la otra y dejar los 20 del docente como prueba final.
   - Dar más peso a la capacidad del país en adaptación (casos como Bélgica) y agregar a la matriz la concordancia a ±1 nivel y la kappa ponderada.